

Test Environment Setup & Validation Report

Project: CTI ADP Software

Report Date: 2026-05-04

Report Version: 1.0

Status: Setup Instructions & Validation Procedures

EXECUTIVE SUMMARY

This report provides a step-by-step guide to build and configure the test environment for all CTI ADP Software projects according to testing requirements defined in TEST_STRATEGY.md. It includes:

-  Prerequisites verification
-  Environment setup for 4 projects (Frontend, Backend, Mobile, Desktop)
-  Database initialization with Docker
-  Configuration & validation at each step
-  Test execution to verify setup
-  Compliance checklist against TEST_STRATEGY.md requirements

PART 1: PREREQUISITES VERIFICATION

Step 1.1: Verify Operating System & Hardware

Requirement: 4+ cores, 8GB+ RAM, 50GB+ SSD

Test Command:

```
# Windows PowerShell
$cpu = (Get-WmiObject -Class Win32_Processor).NumberOfLogicalProcessors
$ram = [math]::Round((Get-WmiObject -Class Win32_ComputerSystem).TotalPhysicalMemory / 1GB)
$disk = [math]::Round((Get-Volume -DriveLetter C).SizeRemaining / 1GB)

Write-Host "CPU Cores: $cpu (Required: 4+)"
Write-Host "RAM GB: $ram (Required: 8+)"
Write-Host "Disk GB Available: $disk (Required: 50+)"
```

Expected Output:

```
CPU Cores: 8 (Required: 4+)
RAM GB: 16 (Required: 8+)
Disk GB Available: 250 (Required: 50+)
```

Status:  Pass /  Fail

Step 1.2: Verify Git Installation

Requirement: Git 2.40+

```
git --version
```

Expected Output:

```
git version 2.40.0 (or higher)
```

If Not Installed:

```
# Windows (choco)
choco install git
```

```
# Or download: https://git-scm.com/download/win
```

Status:  Pass /  Fail

Step 1.3: Verify Repository Cloned & Updated

```
cd d:/dev-js/cti-adp-software
git status
git log --oneline -1
```

Expected Output:

```
On branch main
Your branch is up to date with 'origin/main'.
```

Status:  Pass /  Fail

Step 1.4: Verify Node.js & npm Installation

Requirement: Node.js 18.x, npm 9.x

```
node --version
npm --version
```

Expected Output:

```
v18.19.0 (or v18.x.x)
9.8.1 (or 9.x.x)
```

If Not Installed:

```
# Windows (choco)
choco install nodejs

# Or download: https://nodejs.org/en/download/
```

Status:  Pass /  Fail

Step 1.5: Verify Java Installation

Requirement: Java 17+

```
java --version
```

Expected Output:

```
openjdk 17.0.1 2021-10-19  
OpenJDK Runtime Environment (build 17.0.1+12-39)
```

If Not Installed:

```
# Windows (choco)  
choco install openjdk  
  
# Set JAVA_HOME  
$env:JAVA_HOME = "C:\Program Files\Java\jdk-17"
```

Status:  Pass /  Fail

Step 1.6: Verify Maven Installation

Requirement: Maven 3.8+

```
mvn --version
```

Expected Output:

```
Apache Maven 3.8.1
```

If Not Installed:

```
# Windows (choco)  
choco install maven
```

Status:  Pass /  Fail

Step 1.7: Verify Docker Installation

Requirement: Docker for database container

```
docker --version
docker ps
```

Expected Output:

```
Docker version 24.0.0 (or higher)
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS
```

If Not Installed:

```
# Download Docker Desktop: https://www.docker.com/products/docker-desktop
# Install and restart system
```

Status:  Pass /  Fail

PART 2: DATABASE SETUP & INITIALIZATION

Step 2.1: Start PostgreSQL Database Container

Location: e-commerce/docker-compose.yml

```
cd d:/dev-js/cti-adp-software/e-commerce
docker-compose up -d db
```

Verification:

```
# Check if container is running
docker ps | findstr cti-postgres-db

# Expected: Container should appear in list
```

Expected Output:

CONTAINER ID	IMAGE	PORTS
a1b2c3d4e5f6	postgres:15-alpine	5432/tcp, 0.0.0.0:5433->5432/tcp

Check Container Logs:

```
docker logs cti-postgres-db
```

```
# Should see: "database system is ready to accept connections"
```

Status:  Pass /  Fail

Step 2.2: Verify Database Connection

Connection Test:

```
# Install psql if needed (comes with PostgreSQL installation)
# Windows: Use pgAdmin4 or psql from PostgreSQL bin directory

# Connection string
$ConnectionString = "postgresql://cti_user:cti_password_dev@localhost:5433/cti_ecommerce"

# Test using Docker exec
docker exec cti-postgres-db psql -U cti_user -d cti_ecommerce -c "SELECT 1 as connection_test"
```

Expected Output:

```
connection_test
-----
               1
(1 row)
```

Alternative Test (using psql locally):

```
psql -h localhost -p 5433 -U cti_user -d cti_ecommerce -c "SELECT version()"
```

Status:  Pass /  Fail

Step 2.3: Verify Database Schema Initialization

```
# Check tables created
docker exec cti-postgres-db psql -U cti_user -d cti_ecommerce -c "\dt"
```

Expected Output:

List of relations			
Schema	Name	Type	Owner
public	products	table	cti_user
public	users	table	cti_user
public	orders	table	cti_user
public	cart_items	table	cti_user
...			

Check if data seeded:

```
docker exec cti-postgres-db psql -U cti_user -d cti_ecommerce -c "SELECT COUNT(*) FROM products"
```



Expected Output:

```
count
-----
      10
(1 row)
```

Status:  Pass /  Fail

Step 2.4: Database Environment Summary

Property	Value	Status
Container	cti-postgres-db	✔ Running
Image	postgres:15-alpine	✔ Ready
Host Port	5433	✔ Available
Database	cti_ecommerce	✔ Created
User	cti_user	✔ Created
Tables	4+	✔ Initialized
Test Data	10+ products	✔ Seeded

PART 3: FRONTEND ENVIRONMENT SETUP

Step 3.1: Navigate & Install Dependencies

```
cd d:/dev-js/cti-adp-software/e-commerce/frontend
npm install
```

Verification:

```
# Check node_modules created
if (Test-Path "node_modules") { Write-Host "✔ Dependencies installed" }

# Check package-lock.json updated
if (Test-Path "package-lock.json") { Write-Host "✔ Lock file present" }
```

Expected Behavior:

- ✔ node_modules directory created (~500MB)
- ✔ package-lock.json generated
- ✔ No installation errors

Status: ✔ Pass / ✖ Fail

Step 3.2: Create Environment File

File: e-commerce/frontend/.env.test

```
VITE_API_BASE_URL=http://localhost:3000/api  
VITE_ENV=test
```

Verification:

```
if (Test-Path ".env.test") { Write-Host "✅ .env.test created" }
```

Status: ✅ Pass / ❌ Fail

Step 3.3: TypeScript Type Checking

```
npm run type-check
```

Expected Output:

🌟 Done in 2.34s

If Errors Occur:

```
src/components/Cart.tsx:12:5 - error TS2339: Property 'count' does not exist...
```

Resolution: Fix TypeScript errors before proceeding.

Status: ✅ Pass / ❌ Fail

Step 3.4: Linting Check

```
npm run lint
```

Expected Output:

🌟 Done in 1.23s

If Errors Occur: Fix linting issues (eslint)

Status:  Pass /  Fail

Step 3.5: Run Frontend Tests

```
npm test -- --run --coverage
```

Expected Output:

- ✓ CartContext.test.tsx (5 tests)
- ✓ AddToCartButton.test.tsx (3 tests)
- ✓ useCart.test.tsx (2 tests)

Test Files 3 passed (3)
Tests 10 passed (10)

Coverage: 85.2% (meets target of 75%+)

Metrics to Verify:

Metric	Target	Result	Status
Tests Pass Rate	100%	?	 / 
Coverage	75%+	?	 / 
Type Errors	0	?	 / 
Lint Errors	0	?	 / 

Status:  Pass /  Fail

Step 3.6: Verify Development Server Starts

```
npm run dev &  
# Let it run for 5 seconds, then Ctrl+C to stop  
  
# Expected: Vite v5.0.0 running at http://localhost:5173/
```

Status:  Pass /  Fail

Step 3.7: Frontend Environment Summary

Component	Status	Details
Dependencies		npm install successful
.env.test		API URL configured
Type Check		No TypeScript errors
Lint		ESLint pass
Tests		10/10 passing, 85%+ coverage
Dev Server		Runs on port 5173

Frontend Ready:  YES

PART 4: BACKEND ENVIRONMENT SETUP

Step 4.1: Navigate & Install Dependencies

```
cd d:/dev-js/cti-adp-software/e-commerce/backend  
npm install
```

Verification:

```
if (Test-Path "node_modules") { Write-Host " Dependencies installed" }
```

Status:  Pass /  Fail

Step 4.2: Generate Prisma Client

```
npm run prisma:generate
```

Expected Output:

✓ Generated Prisma Client (5.8.0) in 1.23s

Verification:

```
if (Test-Path "node_modules/.prisma/client") { Write-Host " Prisma client generated" }
```

Status:  Pass /  Fail

Step 4.3: Verify .env Configuration

File: e-commerce/backend/.env

```
NODE_ENV=development
BACKEND_PORT=3000
DATABASE_URL=postgresql://cti_user:cti_password_dev@localhost:5433/cti_ecommerce?schema=public
JWT_SECRET=my_super_secret_key_here_make_it_long_and_secure
```

Verification:

```
# Check file exists and contains required variables
$envContent = Get-Content ".env" -Raw
if ($envContent -match "DATABASE_URL" -and $envContent -match "JWT_SECRET") {
    Write-Host " .env properly configured"
}
```

Status:  Pass /  Fail

Step 4.4: Test Database Connection via Prisma

```
npm run prisma:studio  
# This opens a visual database explorer  
# Close after verifying connection works (port 5555)
```

Alternative Direct Test:

```
# Run a simple test that queries the database  
npm test -- --testNamePattern="database"
```

Expected: Query succeeds and returns data

Status:  Pass /  Fail

Step 4.5: TypeScript Type Checking

```
npm run type-check
```

Expected Output:

 Done in 3.45s

Status:  Pass /  Fail

Step 4.6: Linting Check

```
npm run lint
```

Expected Output:

 Done in 2.10s

Status:  Pass /  Fail

Step 4.7: Run Backend Tests

```
npm test -- --coverage --maxWorkers=1
```

Environment Variable for Tests:

```
$env:DATABASE_URL = "postgresql://cti_user:cti_password_dev@localhost:5433/cti_ecommerce?schema=public"
npm test -- --coverage
```

Expected Output:

```
PASS  src/application/use-cases/LoginClientUseCase.test.ts
PASS  src/application/use-cases/GetProductsUseCase.test.ts
PASS  src/presentation/controllers/AuthController.test.ts
...

Test Suites: 8 passed (8)
Tests:      45 passed (45)

Coverage: 90.5% (meets target of 90%+)
```

Metrics to Verify:

Metric	Target	Result	Status
Tests Pass Rate	100%	?	 / 
Coverage	90%+	?	 / 
Type Errors	0	?	 / 
Lint Errors	0	?	 / 

Status:  Pass /  Fail

Step 4.8: Verify Development Server Starts

```
npm run dev &  
# Let it run for 5 seconds, then Ctrl+C to stop  
  
# Expected: "Backend running on http://localhost:3000"
```

Status:  Pass /  Fail

Step 4.9: Backend Environment Summary

Component	Status	Details
Dependencies		npm install successful
Prisma		Client generated, DB connected
.env		DATABASE_URL & JWT_SECRET set
Type Check		No TypeScript errors
Lint		ESLint pass
Tests		45/45 passing, 90%+ coverage
Dev Server		Runs on port 3000

Backend Ready:  YES

PART 5: MOBILE ENVIRONMENT SETUP

Step 5.1: Set Android SDK Path

```
# Windows PowerShell
$env:ANDROID_HOME = "$env:LOCALAPPDATA\Android\sdk"

# Verify
$env:ANDROID_HOME
```

Expected Output:

```
C:\Users\USERNAME\AppData\Local\Android\sdk
```

Status:  Pass /  Fail

Step 5.2: Verify Gradle Installation

```
cd d:/dev-js/cti-adp-software/mobile
./gradlew --version
```

Expected Output:

```
Gradle 8.2
```

Status:  Pass /  Fail

Step 5.3: Build Mobile Project

```
./gradlew clean build
```

Expected Output:

BUILD SUCCESSFUL in 2m 34s

If Build Fails:

```
# Check for common issues
./gradlew --help
# Verify Android SDK components are installed
```

Status:  Pass /  Fail

Step 5.4: Run Mobile Unit Tests

```
./gradlew test
```

Expected Output:

BUILD SUCCESSFUL in 1m 45s

```
Test Summary:
  Total Tests: 8
  Passed: 8
  Failed: 0
  Skipped: 0
```

Coverage:

```
./gradlew testDebugUnitTestCoverage
```

Expected Coverage: 70%+

Metrics to Verify:

Metric	Target	Result	Status
Tests Pass Rate	100%	?	 / 
Coverage	70%+	?	 / 

Status:  Pass /  Fail

Step 5.5: Mobile Environment Summary

Component	Status	Details
ANDROID_HOME		Path set correctly
Gradle		Version 8.2 installed
Build		Build successful
Tests		8/8 passing, 70%+ coverage

Mobile Ready:  YES

PART 6: DESKTOP ENVIRONMENT SETUP

Step 6.1: Verify Maven Installation

```
mvn --version
```

Expected Output:

```
Apache Maven 3.8.1
```

Status:  Pass /  Fail

Step 6.2: Clean Build Desktop Project

```
cd d:/dev-js/cti-adp-software/desktop
mvn clean compile
```

Expected Output:

[INFO] BUILD SUCCESS

Status:  Pass /  Fail

Step 6.3: Run Desktop Tests

```
mvn test
```

Expected Output:

[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0

[INFO] BUILD SUCCESS

Metrics to Verify:

Metric	Target	Result	Status
Tests Run	12	?	 / 
Failures	0	?	 / 
Errors	0	?	 / 

Status:  Pass /  Fail

Step 6.4: Generate Coverage Report

```
mvn clean test jacoco:report
```

View Report:

```
# Report generated at: target/site/jacoco/index.html
# Open in browser for visual coverage analysis
```

Expected Coverage: 80%+

Status:  Pass /  Fail

Step 6.5: Build Executable JAR

```
mvn clean package
```

Expected Output:

```
[INFO] Building jar: target/stock-management-1.0-SNAPSHOT.jar
```

```
[INFO] BUILD SUCCESS
```

Verify Artifact:

```
if (Test-Path "target/stock-management-1.0-SNAPSHOT.jar") {  
    Write-Host " JAR file created"  
}
```

Status:  Pass /  Fail

Step 6.6: Desktop Environment Summary

Component	Status	Details
Maven		Version 3.8+ installed
Compile		Project compiles
Tests		12/12 passing, 80%+ coverage
Package		Executable JAR created

Desktop Ready:  YES

PART 7: CROSS-PROJECT INTEGRATION TEST

Step 7.1: Start All Services

Terminal 1 - Backend:

```
cd e-commerce/backend
npm run dev
# Runs on http://localhost:3000
```

Terminal 2 - Frontend:

```
cd e-commerce/frontend
npm run dev
# Runs on http://localhost:5173
```

Terminal 3 - Database (already running in Docker):

```
docker ps | findstr cti-postgres-db
```

Verification:

```
# Test backend health
curl http://localhost:3000/api/health

# Expected: {"status":"ok"}

# Test frontend loading
curl http://localhost:5173
# Expected: HTML response
```

Status:  Pass /  Fail

Step 7.2: Run Integration Tests

```
cd e-commerce/backend
npm test -- --testNamePattern="integration"
```

Expected Output:

- ✓ Frontend → Backend → Database integration (5 tests)
 - ✓ Can create user and login
 - ✓ Can add product to cart
 - ✓ Can create order from cart
 - ✓ Database persists data
 - ✓ JWT authentication works

Status:  Pass /  Fail

Step 7.3: Cross-Project Integration Summary

Integration	Status	Details
Frontend ↔ Backend		API calls work
Backend ↔ Database		Queries execute
Auth Flow		JWT token generation works
Data Persistence		Data survives restarts

Cross-Project Ready:  YES

PART 8: COMPLIANCE VERIFICATION

Test Coverage Report

Project	Layer	Target	Achieved	Status
Backend	Use Cases	90%	?	 / 

Project	Layer	Target	Achieved	Status
	Controllers	85%	?	 / 
	Middleware	90%	?	 / 
Frontend	Context Logic	90%	?	 / 
	Components	75%	?	 / 
Mobile	UI Components	70%	?	 / 
Desktop	Controllers	80%	?	 / 

Code Quality Report

Check	Target	Status
Type Safety	No any types	 / 
Frontend Lint	0 errors	 / 
Backend Lint	0 errors	 / 
All Tests Pass	100%	 / 
No Flaky Tests	0 failures	 / 

TEST_STRATEGY.md Compliance Checklist

Based on TEST_STRATEGY.md Section 3 (Standards):

- ☐ Backend Use Cases: 90%+ coverage achieved
- ☐ Backend Controllers: 85%+ coverage achieved
- ☐ Backend Middleware: 90%+ coverage achieved
- ☐ Frontend Context Logic: 90%+ coverage achieved
- ☐ Frontend Components: 75%+ coverage achieved
- ☐ Mobile UI: 70%+ coverage achieved
- ☐ Desktop UI: 80%+ coverage achieved
- ☐ No TypeScript any types

- ☐ All tests named with pattern: {unit} should {behavior} when {condition}
- ☐ All tests passing locally before CI
- ☐ ESLint/type-check all passing
- ☐ Each test is isolated (no shared state)
- ☐ All assertions present and meaningful

PART 9: ENVIRONMENT READINESS CHECKLIST

Prerequisites (Part 1)

- ☐ OS: 4+ cores, 8GB+ RAM, 50GB+ SSD
- ☐ Git: 2.40+
- ☐ Repository: Cloned & up-to-date
- ☐ Node.js: 18.x installed
- ☐ npm: 9.x installed
- ☐ Java: 17+ installed
- ☐ Maven: 3.8+ installed
- ☐ Docker: Installed & running

Database (Part 2)

- ☐ PostgreSQL container running
- ☐ Database connection verified
- ☐ Schema initialized (4+ tables)
- ☐ Test data seeded (10+ products)
- ☐ All tables populated

Frontend (Part 3)

- ☐ Dependencies installed (node_modules/)
- ☐ .env.test created
- ☐ TypeScript: No errors
- ☐ ESLint: No errors
- ☐ Tests: All passing
- ☐ Coverage: 75%+ achieved
- ☐ Dev server: Starts successfully

Backend (Part 4)

- ☐ Dependencies installed (node_modules/)
- ☐ Prisma client generated
- ☐ .env configured (DATABASE_URL, JWT_SECRET)
- ☐ Database connection: Verified
- ☐ TypeScript: No errors
- ☐ ESLint: No errors
- ☐ Tests: All passing
- ☐ Coverage: 90%+ achieved
- ☐ Dev server: Starts successfully

Mobile (Part 5)

- ☐ ANDROID_HOME set
- ☐ Gradle: 8.2+
- ☐ Build: Successful
- ☐ Tests: All passing
- ☐ Coverage: 70%+ achieved

Desktop (Part 6)

- ☐ Maven: 3.8+
- ☐ Compile: Successful
- ☐ Tests: All passing
- ☐ Coverage: 80%+ achieved
- ☐ Package: JAR created

Integration (Part 7)

- ☐ Frontend ↔ Backend: Connected
- ☐ Backend ↔ Database: Connected
- ☐ Auth flow: Working
- ☐ Data persistence: Verified

TEST_STRATEGY.md Compliance (Part 8)

- ☐ All coverage targets met
- ☐ Code quality standards met

- ☐ Test naming convention followed
- ☐ All tests isolated
- ☐ No flaky tests
- ☐ Ready for CI/CD

PART 10: DEPLOYMENT RECORD

Environment Build Date & Time

Setup Start: 2026-05-04 [HH:MM]

Setup Complete: 2026-05-04 [HH:MM]

Duration: [X hours Y minutes]

Deployed By

Role	Name	Date	Signature
DevOps Engineer	[Name]	2026-05-04	✓
QA Lead	[Name]	2026-05-04	✓
Tech Lead	[Name]	2026-05-04	✓

Verification Sign-Off

- All environment components have been:
- ☒ Successfully installed

• ☒ Properly configured

• ☒ Tested and validated

• ☒ Verified for compliance with TEST_STRATEGY.md

Verified By: [QA Lead Name]

Date: 2026-05-04

Status: ☒ ENVIRONMENT READY FOR TESTING

PART 11: TROUBLESHOOTING REFERENCE

If Any Step Fails

Issue	Cause	Solution
Database Connection Failed	Docker not running	<code>docker-compose up -d db</code>
Tests Timeout	Async issues	Check test for <code>async/await</code>
Port Already in Use	Service already running	Change port or kill process
TypeScript Error	Type mismatch	Run <code>tsc --noEmit</code> to see all errors
Module Not Found	Dependencies not installed	Run <code>npm install</code> again
Build Failure	Java version wrong	Verify <code>java --version</code> shows 17+

Full Troubleshooting: See TEST_ENVIRONMENT.md Section 11

PART 12: NEXT STEPS

After Environment Setup Complete:

1. Run Full Test Suite:

```
# Frontend
cd e-commerce/frontend && npm test -- --run

# Backend
cd e-commerce/backend && npm test -- --coverage

# Mobile
cd mobile && ./gradlew test

# Desktop
cd desktop && mvn test
```

2. Generate Coverage Reports:

```
# All projects covered above  
# Reports ready in: coverage/ and target/site/
```

3. Start Development:

```
# Terminal 1: Backend  
cd e-commerce/backend && npm run dev  
  
# Terminal 2: Frontend  
cd e-commerce/frontend && npm run dev  
  
# Terminal 3: (Optional) Prisma Studio  
cd e-commerce/backend && npm run prisma:studio
```

4. Begin Test Execution:

- Follow TEST_PLAN.md test cases
- Execute manual QA workflows
- Document results

APPENDIX A: Quick Command Reference

Database

docker-compose up -d db

docker ps

docker logs cti-postgres-db

Start DB

Check status

View logs

Frontend

npm test -- --run

npm run test:coverage

npm run dev

Run tests once

Coverage report

Dev server

Backend

npm test -- --coverage

npm run prisma:studio

npm run dev

Run tests with coverage

Visual DB tool

Dev server

Mobile

./gradlew test

./gradlew build

Run tests

Build APK

Desktop

mvn test

mvn clean package

Run tests

Build JAR

APPENDIX B: Port Map

Service	Port	Status
Frontend	5173	Development
Backend API	3000	Development
Database	5433	Container mapped
Prisma Studio	5555	Optional

Document Owner: DevOps/QA Lead

Last Updated: 2026-05-04

Status: Ready for Implementation

Revision: 1.0